

# Tutorial — Pipeline de Dados EONET (NASA)

---

Guia para replicação completa do projeto — Grupo 02

Antonela, Brenda e Ikaro

- [1. Visão geral](#)
- [2. Criar a estrutura no GitHub](#)
- [3. Criar o projeto no Supabase](#)
- [4. Modelar o banco de dados](#)
- [5. Obter as credenciais do Supabase](#)
- [6. Configurar os Secrets no GitHub](#)
- [7. Script de coleta \( `scripts/fetch\_nasa.py` \)](#)
- [8. Workflow de automação](#)
- [9. Painel de dados \( `index.html` \)](#)
- [10. Verificação final](#)
- [11. Estrutura final do repositório](#)

## 1. Visão geral

---

Este guia descreve, passo a passo, como reproduzir o pipeline de dados construído com a API aberta **EONET** (Earth Observatory Natural Event Tracker) da NASA. O fluxo final é:

EONET API → GitHub Actions (coleta agendada) → Supabase (PostgreSQL) → GitHub Pages (painel)

A API EONET disponibiliza eventos naturais em curso — incêndios, tempestades severas, erupções vulcânicas, gelo marinho e lacustre, entre outros — e **não exige chave de acesso**, o que simplifica a etapa de autenticação.

Pré-requisitos: conta no GitHub, conta no Supabase e Python 3 (usado apenas localmente para testes, caso deseje).

## 2. Criar a estrutura no GitHub

---

1. Crie (ou renomeie) uma **Organização** no GitHub com o nome `SN-2026-GRUPO-Nº-NASA`, substituindo `Nº` pelo número do grupo.
2. Dentro dessa organização, crie um **repositório público** com o nome da API escolhida (no caso deste projeto, `EONET`).
3. Em **People** → **Invite member**, adicione o professor responsável como membro da organização.

## 3. Criar o projeto no Supabase

---

1. Crie uma **Organização** no Supabase com o mesmo nome usado no GitHub.
2. Crie um **Projeto** com o padrão `NOME-DA-API-DB` (ex.: `EONET-DB`), definindo a região **South America (São Paulo)**.
3. Em **Settings** → **Team** → **Invite**, adicione o e-mail do professor com permissão

**Developer.**

4. Habilite o **Row Level Security (RLS)** nas tabelas — isso será feito automaticamente pelo script SQL do próximo passo.

## 4. Modelar o banco de dados

---

No projeto Supabase, abra **SQL Editor** → **New query**, cole o conteúdo do arquivo `sql/setup.sql` do repositório e execute (**Run**).

O script cria duas tabelas:

- **eonet\_eventos** — uma linha por evento, com uma constraint `UNIQUE` em `evento_id` (a chave natural do registro). Essa constraint é o que permite fazer *upsert* sem duplicar dados a cada nova coleta.
- **execucoes** — registra o histórico de execuções do pipeline: data de início e fim, registros processados, número de lotes, erros e status ( `concluido` , `erro_parcial` ou `erro_critico` ).

Ambas as tabelas têm **RLS ativo** com uma política de leitura pública ( `SELECT` ), e **GRANTS explícitos**: `SELECT` para os papéis `anon` e `authenticated` , e `ALL` para `service_role` . Isso garante que o painel público só consiga ler os dados, nunca alterá-los, enquanto o pipeline (usando a chave de serviço) tem acesso completo.

Confira no **Table Editor** se as duas tabelas aparecem corretamente antes de prosseguir.

## 5. Obter as credenciais do Supabase

---

Em **Project Settings** → **API**, anote três valores:

| Valor                                    | Uso                                                      |
|------------------------------------------|----------------------------------------------------------|
| Project URL                              | variável <code>SUPABASE_URL</code>                       |
| Secret key ( <code>service_role</code> ) | variável <code>SUPABASE_SERVICE_KEY</code>               |
| Publishable key ( <code>anon</code> )    | usada diretamente no <code>index.html</code> (é pública) |

**Atenção:** a secret key nunca deve aparecer em código, commits ou prints. Se isso acontecer, regenere a chave imediatamente em **Project Settings** → **API Keys**.

## 6. Configurar os Secrets no GitHub

---

No repositório, acesse **Settings** → **Secrets and variables** → **Actions** → **New repository secret** e cadastre:

- `SUPABASE_URL`
- `SUPABASE_SERVICE_KEY`

Esses valores ficam disponíveis apenas durante a execução do workflow, como variáveis de ambiente — nunca aparecem no código-fonte.

## 7. Script de coleta ( `scripts/fetch_nasa.py` )

---

O script localizado em `scripts/fetch_nasa.py` realiza, em sequência:

1. **Consulta** o endpoint `https://eonet.gsfc.nasa.gov/api/v3/events` , com até três tentativas

em caso de falha temporária.

2. **Normaliza** cada evento para uma linha de tabela, extraindo a geometria mais recente e preenchendo `latitude / longitude` apenas quando ela é um ponto (eventos com geometria em polígono mantêm o dado bruto no campo `geometrias`, em JSON).
3. **Deduplica** os registros por `evento_id`, evitando o erro PostgreSQL 21000 (violação de chave única por linhas repetidas no mesmo lote).
4. **Envia os dados em lotes** (200 registros por padrão) via `upsert` na API REST do Supabase, usando `on_conflict=evento_id`.
5. **Registra a execução** na tabela `execucoes`, com o status apropriado, e encerra com `sys.exit(1)` em caso de erro — isso faz o GitHub Actions marcar a execução como falha, tornando o problema visível nos logs.

Variáveis de ambiente opcionais permitem ajustar o comportamento sem alterar o código: `EONET_STATUS` (`all / open / closed`), `EONET_DAYS` (janela de dias) e `BATCH_SIZE`.

## 8. Workflow de automação

---

O arquivo `.github/workflows/update-data.yml` define a execução automatizada:

- `schedule` — executa diariamente via expressão cron.
- `workflow_dispatch` — permite execução manual pela aba *Actions*.
- `permissions: contents: read` — concede apenas o acesso mínimo necessário ao repositório.
- `concurrency` — impede que duas execuções rodem ao mesmo tempo.
- `timeout-minutes` — cancela a execução caso ela trave.

Para testar manualmente: aba **Actions** → **Atualizar dados EONET** → **Run workflow**. Uma execução bem-sucedida aparece com um ícone verde e grava dados reais nas tabelas do Supabase.

## 9. Painel de dados ( `index.html` )

---

O arquivo `index.html`, publicado via **GitHub Pages** (Settings → Pages → branch `main`, pasta raiz), consome diretamente a API REST do Supabase pelo navegador, usando a chave pública (`anon`).

Funcionalidades do painel:

- Lista os eventos em cards, com cor por categoria.
- Dois filtros interativos: por categoria e por status (aberto/fechado).
- Exibe a data da **última atualização**, lida da tabela `execucoes`.
- Aplica a função `escapeHtml()` a todo valor vindo do banco antes de inserir no HTML, prevenindo XSS a partir de dados externos.

Antes de publicar, é necessário preencher `SUPABASE_URL` e `SUPABASE_ANON_KEY` no topo do script da página.

## 10. Verificação final

---

Antes da entrega, confirme:

- ☐ Workflow com execução verde mais recente.
- ☐ Tabela `eonet_eventos` com dados reais no Supabase.
- ☐ Tabela `execucoes` com ao menos um registro de status `concluido`.
- ☐ Painel acessível publicamente, com filtros funcionando.

- ☐ README.md , docs/ai-interaction.md , docs/reflexao.md e docs/apresentacao.pdf completos no repositório.
- ☐ Planilha de reserva atualizada com os links definitivos (colunas G e H).

## 11. Estrutura final do repositório

---

```
EONET/  
├─ index.html  
├─ README.md  
├─ scripts/  
│   └─ fetch_nasa.py  
├─ sql/  
│   └─ setup.sql  
├─ .github/workflows/  
│   └─ update-data.yml  
└─ docs/  
    ├─ tutorial.pdf  
    ├─ ai-interaction.md  
    ├─ reflexao.md  
    └─ apresentacao.pdf
```